

MeloStream

Plateforme de streaming musical

Rapport de projet

Application web full-stack avec Spring Boot, Angular et MySQL

Réalisé par : Ayoub El Houani

Oussama Felouach

Mbarek Ettaleby

Type : Projet web full-stack

Backend : Java 21, Spring Boot

Frontend : Angular, TypeScript

Base de données : MySQL

Année universitaire : 2025–2026

Table des matières

Introduction	3
Contexte	3
Objectifs	3
Démarche de réalisation	4
1 Cahier des charges	5
1.1 Problématique	5
1.2 Besoins identifiés	5
1.2.1 Besoins fonctionnels	5
1.2.2 Besoins non fonctionnels	6
1.3 Solutions proposées	6
2 Conception fonctionnelle	7
2.1 Acteurs	7
2.2 Diagramme de cas d'utilisation	8
2.3 Diagramme de classes	9
2.4 Organisation des données	10
2.5 Scénario de partage d'un titre	10
3 Conception technique	11
3.1 Architecture globale	11
3.2 Technologies utilisées	12
3.3 Structure du projet	12
3.4 Sécurité	13
3.5 Endpoints REST principaux	13
3.6 Documentation OpenAPI et Swagger	14
4 Réalisation	15
4.1 Fonctionnalités développées	15

4.1.1	Authentification	15
4.1.2	Recherche musicale	15
4.1.3	Lecture	15
4.1.4	Favoris	16
4.1.5	Playlists	16
4.1.6	Partage	16
4.1.7	Administration	16
4.1.8	Documentation API	16
4.2	Captures d'écran	17
4.3	Comptes de démonstration	23
5	Installation et lancement	24
5.1	Prérequis	24
5.2	Configuration de la base	24
5.3	Lancement du backend	24
5.4	Lancement du frontend	25
5.5	Activation optionnelle de Jamendo	25
6	Vérification	26
6.1	Tests backend	26
6.2	Tests frontend	26
6.3	Build frontend	26
	Conclusion	27

Introduction

Contexte

Les plateformes musicales occupent aujourd’hui une place importante dans les usages numériques. Elles permettent de rechercher des chansons, découvrir des artistes, organiser ses préférences et partager facilement des titres avec d’autres utilisateurs.

Le projet MeloStream consiste à développer une application web de streaming musical dans un contexte pédagogique. L’application propose une interface moderne côté client, une API REST sécurisée côté serveur et une base de données permettant de conserver les utilisateurs, les sessions et les titres favoris.

Le système utilise des services musicaux externes pour récupérer les chansons, les artistes, les albums, les pochettes et les liens audio. Deezer est utilisé par défaut pour les extraits audio, tandis que Jamendo peut être activé afin de proposer des titres libres ou licenciés.

Objectifs

Les objectifs du projet sont :

- créer une application web complète avec une séparation claire entre frontend et backend ;
- permettre l’inscription, la connexion, la déconnexion et la gestion du profil utilisateur ;
- proposer un catalogue musical consultable par recherche ou par humeur ;
- intégrer un lecteur audio sans lancer automatiquement la musique après la connexion ;
- permettre la sauvegarde des titres favoris ;
- permettre la création de playlists et l’organisation des titres ;
- ajouter une fonctionnalité de partage de titre par lien ;
- fournir une interface d’administration protégée par rôle ;
- produire une documentation claire pour comprendre, lancer et maintenir le projet.

Démarche de réalisation

La réalisation du projet a été organisée de manière progressive. Une première étape a consisté à définir les besoins principaux et les entités nécessaires : utilisateurs, jetons de connexion, favoris, playlists et titres musicaux. Le backend a ensuite été mis en place pour exposer une API REST sécurisée, avant d'intégrer l'interface Angular et les appels vers les services musicaux externes.

Le développement a suivi une logique fonctionnelle : chaque fonctionnalité visible dans l'interface correspond à un ensemble clair de routes backend, de services métier, de DTO et de méthodes côté frontend. Cette approche facilite la maintenance, car une évolution sur les favoris, les playlists ou l'administration reste localisée dans le domaine concerné.

Chapitre 1

Cahier des charges

1.1 Problématique

La problématique principale est de concevoir une plateforme musicale simple, ergonomique et sécurisée, capable de relier une interface Angular à une API Spring Boot et à une base de données MySQL. Le projet doit aussi communiquer avec des API musicales externes sans rendre le code difficile à maintenir.

L'application doit donc répondre à trois exigences majeures :

- fournir une expérience utilisateur fluide ;
- protéger les fonctionnalités privées par authentification ;
- garder une architecture technique claire et évolutive.

1.2 Besoins identifiés

1.2.1 Besoins fonctionnels

Référence	Description
BF01	Créer un compte utilisateur.
BF02	Se connecter et se déconnecter.
BF03	Consulter et modifier son profil.
BF04	Rechercher des titres par mot-clé, artiste, album ou humeur.
BF05	Écouter un titre depuis un lecteur intégré.
BF06	Ajouter un titre aux favoris.
BF07	Supprimer un titre des favoris.
BF08	Créer, renommer et supprimer une playlist.
BF09	Ajouter ou retirer des titres d'une playlist.
BF10	Partager un titre avec un lien direct.
BF11	Charger automatiquement le titre ciblé lorsqu'un lien partagé est ouvert.

BF12	Accéder à un espace administrateur pour gérer les utilisateurs.
------	---

1.2.2 Besoins non fonctionnels

Référence	Description
BNF01	L'interface doit être responsive et lisible.
BNF02	Les mots de passe doivent être stockés sous forme de hash.
BNF03	Les endpoints privés doivent être protégés par jeton.
BNF04	Les rôles doivent limiter l'accès à l'administration.
BNF05	Le code doit être organisé par domaine fonctionnel.
BNF06	La base de données doit être initialisée automatiquement en local.
BNF07	Le projet doit être testable et documenté.

1.3 Solutions proposées

La solution proposée repose sur :

- une application Angular pour l'interface utilisateur ;
- une API Spring Boot pour la logique métier et la sécurité ;
- une base MySQL pour la persistance ;
- une intégration Deezer/Jamendo pour les données musicales ;
- une authentification stateless par jeton ;
- une interface administrateur accessible seulement au rôle **ADMIN**.

Cette solution permet de séparer les responsabilités. Le frontend gère l'affichage, les formulaires, les interactions utilisateur et le lecteur audio. Le backend contrôle les règles métier, la persistance et la sécurité. La base MySQL conserve les données propres à l'application, tandis que les informations musicales sont récupérées à la demande depuis des services externes.

Chapitre 2

Conception fonctionnelle

2.1 Acteurs

Acteur	Rôle dans le système
Visiteur	Peut créer un compte ou se connecter.
Utilisateur	Peut rechercher, écouter, sauvegarder, organiser et partager des titres.
Administrateur	Peut gérer les utilisateurs et consulter les statistiques.
API musicale externe	Fournit les titres, artistes, albums, images et audios.

2.2 Diagramme de cas d'utilisation

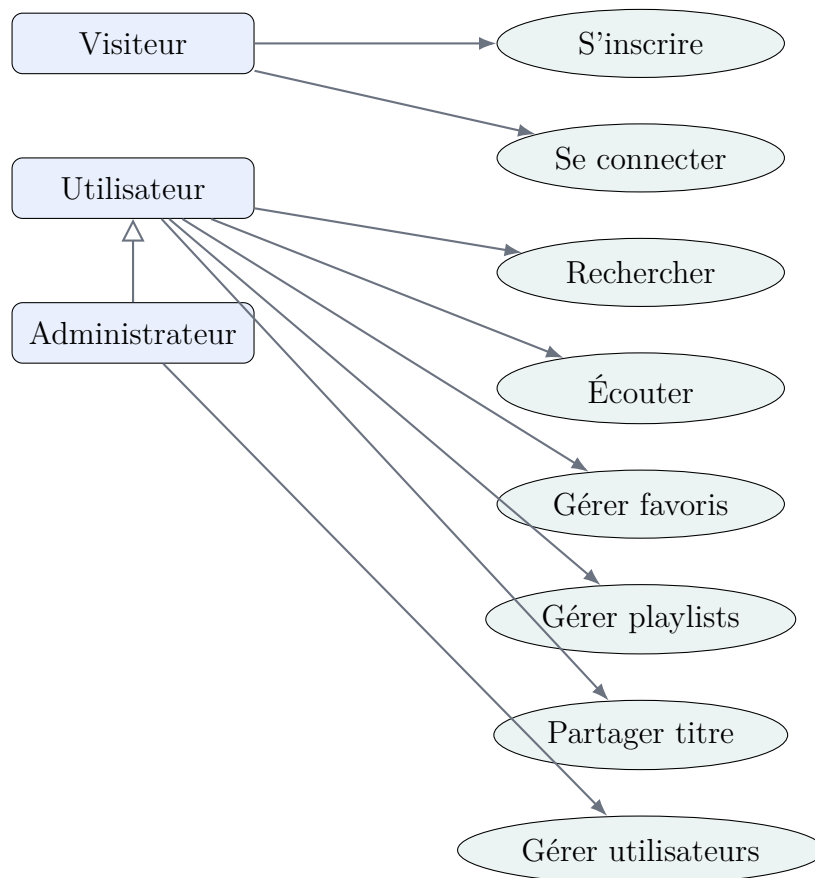


FIGURE 2.1 – Diagramme de cas d'utilisation simplifié

Explication du diagramme :

- Le diagramme présente les principales interactions entre les acteurs et l'application.
- Le visiteur peut seulement créer un compte ou se connecter.
- Après authentification, l'utilisateur accède aux fonctionnalités musicales : recherche, écoute, favoris, playlists et partage.
- La flèche de généralisation indique que l'administrateur est un utilisateur spécialisé.
- L'administrateur conserve les cas d'utilisation d'un utilisateur connecté et ajoute la gestion des utilisateurs.
- Cette représentation distingue les fonctionnalités publiques, les fonctionnalités privées et les fonctionnalités d'administration.

2.3 Diagramme de classes

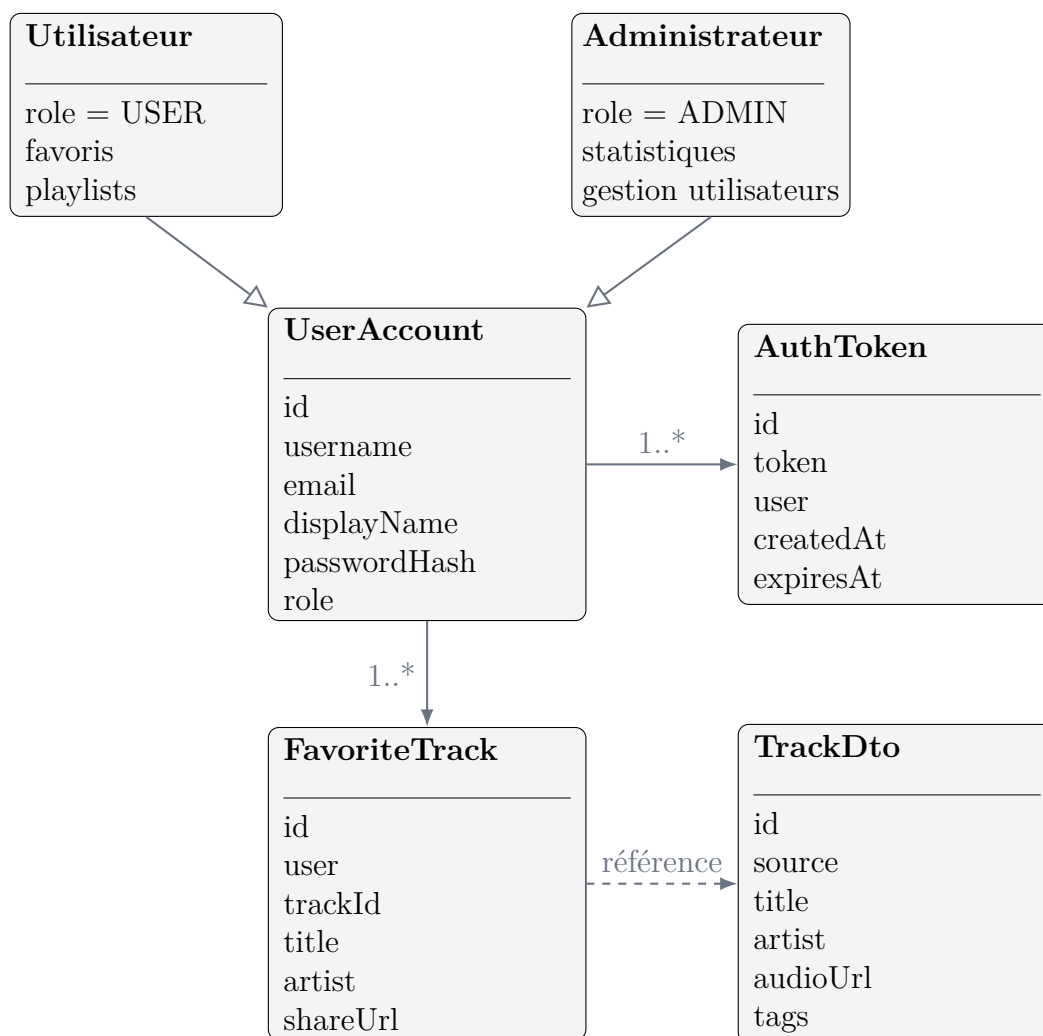


FIGURE 2.2 – Diagramme de classes principal

Explication du diagramme :

- Le diagramme décrit les entités principales utilisées par le backend.
- **UserAccount** représente le compte utilisateur avec son identifiant, son profil, son mot de passe hashé et son rôle.
- **AuthToken** conserve les jetons de session liés à un utilisateur pour authentifier les requêtes privées.
- La généralisation montre que **Utilisateur** et **Administrateur** sont deux spécialisations de **UserAccount**.
- Dans le code Java, cette spécialisation est portée par le champ `role` : `USER` pour les fonctions musicales et `ADMIN` pour l'administration.
- **FavoriteTrack** sauvegarde les titres favoris d'un utilisateur, tandis que **TrackDto** représente un titre normalisé venant d'une API musicale externe.
- Les classes **Playlist** et **PlaylistTrack** complètent le modèle pour organiser les chansons en listes personnalisées.

2.4 Organisation des données

Les données de l'application sont réparties entre les données internes et les données externes. Les données internes concernent les comptes utilisateurs, les jetons d'authentification, les favoris et les playlists. Elles sont stockées dans MySQL afin de rester disponibles après le redémarrage de l'application.

Les données musicales, comme le titre, l'artiste, l'album, la pochette ou l'extrait audio, proviennent principalement des API Deezer et Jamendo. Le backend les transforme en objets `TrackDto` pour fournir au frontend un format unique, quelle que soit la source musicale utilisée. Cette normalisation évite de faire dépendre l'interface Angular du format exact de chaque API externe.

2.5 Scénario de partage d'un titre

Le partage est une fonctionnalité importante de l'application. Elle permet à un utilisateur de copier un lien vers un titre et à un autre utilisateur d'accéder directement au même titre.

1. L'utilisateur clique sur le bouton **Partager**.
2. Le frontend construit une URL au format `?song=source:id`.
3. Exemple : <http://localhost:4200/?song=deezer:3135556>.
4. Un autre utilisateur ouvre ce lien.
5. Après connexion, le frontend appelle l'API `/api/tracks/{source}/{trackId}`.
6. Le titre est chargé dans la zone de lecture rapide.
7. La musique ne démarre pas automatiquement.

Chapitre 3

Conception technique

3.1 Architecture globale

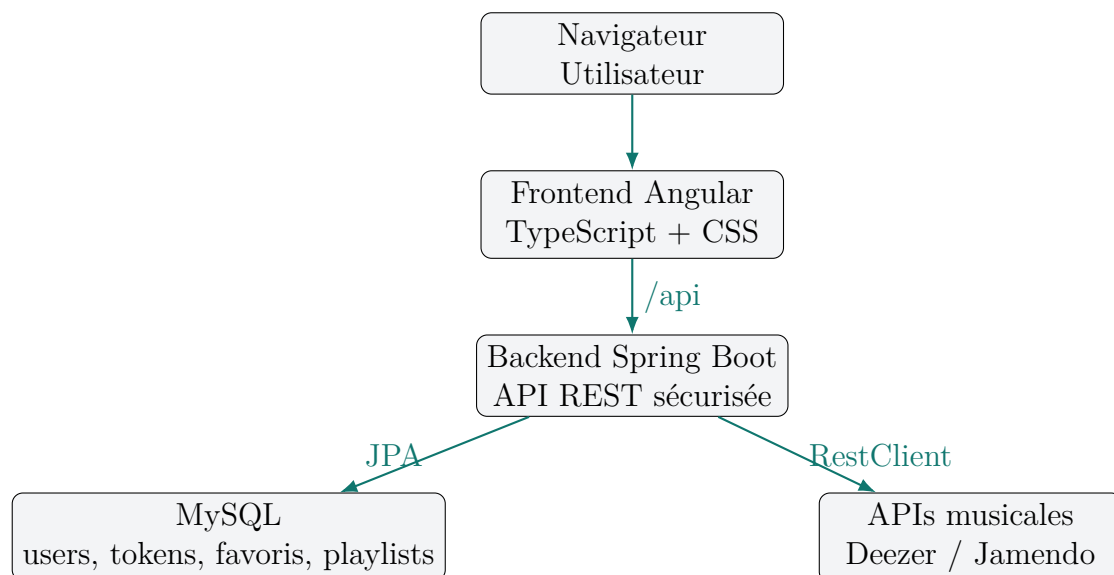


FIGURE 3.1 – Architecture technique de MeloStream

Explication du diagramme :

- Le diagramme montre le chemin suivi par les données entre l'utilisateur et les couches techniques du système.
- Le navigateur communique avec le frontend Angular, responsable de l'interface, des formulaires et des interactions utilisateur.
- En développement local, Angular redirige les requêtes **/api** vers le backend Spring Boot grâce au proxy.
- Le backend applique les règles métier, contrôle la sécurité et sert de point central entre le frontend, MySQL et les API musicales.
- MySQL stocke les données internes : utilisateurs, jetons, favoris et playlists.
- Les API Deezer et Jamendo fournissent les données musicales externes, comme les titres, artistes, pochettes et extraits audio.
- Les échanges entre les couches utilisent le format JSON, avec des DTO pour fournir au frontend des données normalisées.

3.2 Technologies utilisées

Couche	Technologies
Frontend	Angular 21, TypeScript, RxJS, CSS responsive, @lucide/angular.
Backend	Java 21, Spring Boot 4.0.6, Spring Web MVC, Spring Security, Spring Data JPA.
Base de données	MySQL avec accès possible via phpMyAdmin.
Documentation API	Springdoc OpenAPI 3.0.3 et Swagger UI.
APIs externes	Deezer public API et Jamendo API optionnelle.
Tests	Spring Boot Test, H2 pour les tests backend, Angular/- Vitest côté frontend.

3.3 Structure du projet

```
project web/  
  backend/  
    src/main/java/com/musicstream/  
      admin/  
      auth/  
      config/  
      deezer/  
      favorites/  
      jamendo/  
      music/  
      security/  
    src/main/resources/application.yml  
    pom.xml  
  frontend/  
    src/app/  
      admin-api.ts  
      app.css  
      app.html  
      app.spec.ts  
      app.ts  
      auth-api.ts  
      favorite-api.ts  
      music-api.ts  
    package.json  
    proxy.conf.json  
  rapport/  
    rapport.tex  
    rapport.pdf  
  README.md
```

3.4 Sécurité

La sécurité est assurée par Spring Security. L'application utilise un modèle stateless : après connexion, le backend génère un jeton stocké en base dans la table `auth_tokens`. Le frontend transmet ensuite ce jeton pour accéder aux endpoints protégés.

- `/api/auth/login` et `/api/auth/register` sont publics.
- `/swagger-ui.html`, `/swagger-ui/**` et `/v3/api-docs/**` sont publics pour consulter la documentation API.
- Les endpoints `/api/**` nécessitent une authentification.
- Les endpoints `/api/admin/**` nécessitent le rôle `ADMIN`.
- Les mots de passe sont hashés avant stockage.
- Les règles CORS autorisent le frontend local sur le port 4200.

Lorsqu'un utilisateur se connecte, le backend vérifie l'email et le mot de passe, puis crée un jeton temporaire valable plusieurs jours. Ce jeton est envoyé au frontend et stocké localement. Pour les requêtes privées, Angular l'ajoute dans l'en-tête `Authorization`. Le filtre de sécurité du backend lit ce jeton, retrouve l'utilisateur associé et autorise ou refuse l'accès selon son rôle.

Cette stratégie reste simple à utiliser dans un projet pédagogique tout en respectant les principes essentiels : les mots de passe ne circulent qu'au moment de la connexion, les routes sensibles sont protégées et les actions administrateur sont séparées des actions utilisateur.

3.5 Endpoints REST principaux

Méthode	Endpoint	Description
POST	<code>/api/auth/register</code>	Créer un compte.
POST	<code>/api/auth/login</code>	Se connecter.
GET	<code>/api/auth/me</code>	Récupérer l'utilisateur connecté.
PUT	<code>/api/auth/profile</code>	Modifier le profil.
POST	<code>/api/auth/logout</code>	Se déconnecter.
GET	<code>/api/tracks</code>	Rechercher des titres.
GET	<code>/api/tracks/{source}/{trackId}</code>	Charger un titre partagé.
GET	<code>/api/favorites</code>	Lister les favoris.
POST	<code>/api/favorites</code>	Ajouter un favori.
DELETE	<code>/api/favorites/{trackId}</code>	Supprimer un favori.
GET	<code>/api/playlists</code>	Lister les playlists.
POST	<code>/api/playlists</code>	Créer une playlist.

PUT	/api/playlists/{playlistId}	Renommer une playlist.
DELETE	/api/playlists/{playlistId}	Supprimer une playlist.
POST	/api/playlists/{playlistId}/tracks	Ajouter un titre à une playlist.
DELETE	/api/playlists/{playlistId}/tracks/{source}/{trackId}	Retirer un titre d'une playlist.
GET	/api/admin/stats	Consulter les statistiques.
GET	/api/admin/users	Lister les utilisateurs.
PUT	/api/admin/users/{userId}/role	Modifier un rôle.
DELETE	/api/admin/users/{userId}	Supprimer un utilisateur.

3.6 Documentation OpenAPI et Swagger

Le backend intègre Springdoc OpenAPI afin de générer automatiquement une documentation interactive à partir des controllers REST. La page Swagger UI est accessible localement après le démarrage du backend :

```
http://localhost:8080/swagger-ui.html
```

La spécification OpenAPI JSON est disponible à l'adresse suivante :

```
http://localhost:8080/v3/api-docs
```

Cette interface permet de consulter les endpoints, les schémas DTO et d'utiliser le bouton **Authorize** avec le jeton d'authentification retourné par l'endpoint de connexion.

Chapitre 4

Réalisation

4.1 Fonctionnalités développées

4.1.1 Authentification

L'utilisateur peut créer un compte, se connecter et se déconnecter. Après une connexion réussie, un jeton est utilisé pour sécuriser les requêtes suivantes. Les comptes de démonstration sont créés automatiquement au démarrage.

Le formulaire d'authentification vérifie les informations saisies avant l'envoi au backend. Après une connexion réussie, l'utilisateur accède au tableau de bord musical avec son profil chargé depuis l'endpoint `/api/auth/me`. La déconnexion supprime le jeton local et invalide les jetons actifs côté serveur.

4.1.2 Recherche musicale

Le catalogue permet de rechercher des titres par mot-clé. L'utilisateur peut chercher un titre, un artiste ou un album. Le backend interroge les services musicaux externes et retourne des objets normalisés au frontend.

Les résultats affichent les informations utiles à l'écoute et à l'organisation des titres : nom de la chanson, artiste, album, image, durée et source. Cette présentation donne à l'utilisateur assez de contexte pour lancer la lecture, ajouter le titre aux favoris, le placer dans une playlist ou le partager.

4.1.3 Lecture

Chaque titre peut être lancé depuis l'interface. Une attention particulière a été portée au comportement après connexion : l'application ne lance pas la musique automatiquement.

Le lecteur reste donc contrôlé par l'utilisateur. Lorsqu'un titre est choisi, l'application met à jour la zone de lecture rapide et utilise l'URL audio fournie par l'API musicale. Si aucun extrait n'est disponible, l'interface peut continuer à afficher le titre sans déclencher de lecture.

4.1.4 Favoris

Un utilisateur connecté peut sauvegarder un titre dans ses favoris et le retrouver plus tard. Les favoris sont persistés dans la base MySQL.

Chaque favori est rattaché au compte connecté. Cela permet d'avoir une bibliothèque personnelle différente pour chaque utilisateur. La contrainte d'unicité évite d'ajouter plusieurs fois le même titre dans les favoris d'un même compte.

4.1.5 Playlists

L'utilisateur peut créer des playlists personnalisées, les renommer, les supprimer et y ajouter des chansons depuis le catalogue ou la bibliothèque. Les titres d'une playlist peuvent ensuite être écoutés, partagés ou retirés.

Les playlists complètent les favoris en ajoutant une organisation par listes. Elles sont utiles pour regrouper des chansons par thème, humeur ou usage. Le backend vérifie que l'utilisateur connecté est bien propriétaire de la playlist avant d'autoriser les modifications.

4.1.6 Partage

Chaque chanson possède un bouton de partage. Le lien généré contient la source et l'identifiant du titre. Exemple :

```
http://localhost:4200/?song=deezer:3135556
```

Lorsqu'un autre utilisateur ouvre ce lien, le titre est chargé après authentification.

Le partage ne stocke pas une copie complète du titre dans l'URL. Le lien contient seulement la source musicale et l'identifiant externe. Le backend peut alors recharger les informations du morceau via l'endpoint dédié et renvoyer au frontend un `TrackDto` complet.

4.1.7 Administration

Les comptes administrateurs peuvent consulter les statistiques, lister les utilisateurs, modifier les rôles et supprimer un utilisateur autre que leur propre compte.

L'espace administrateur est masqué pour les utilisateurs simples et protégé côté backend. Les statistiques donnent une vue rapide sur le nombre d'utilisateurs, de favoris et de playlists. Les actions sensibles, comme la suppression d'un compte ou la modification d'un rôle, passent par les endpoints `/api/admin/**`.

4.1.8 Documentation API

Swagger UI fournit une documentation interactive des endpoints REST du backend. Elle facilite la vérification des routes, des paramètres et des modèles échangés entre le frontend Angular et l'API Spring Boot.

4.2 Captures d'écran

Les captures suivantes ont été réalisées sur l'application locale. Elles présentent les principaux écrans développés : authentification, inscription, catalogue, recherche, partage, favoris, playlists, paramètres, administration et documentation Swagger.

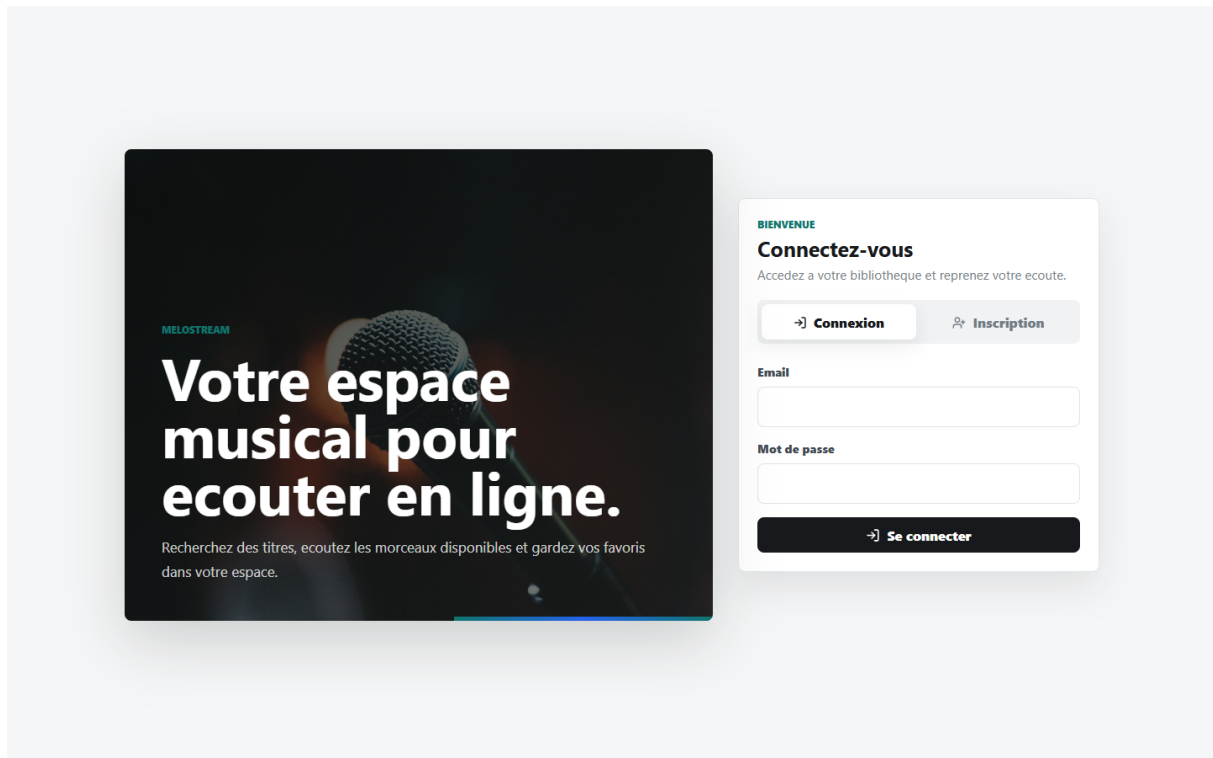


FIGURE 4.1 – Page d'authentification

Explication de la figure : cette page permet à l'utilisateur de se connecter à son compte. Elle constitue le point d'entrée vers les fonctionnalités privées de l'application.

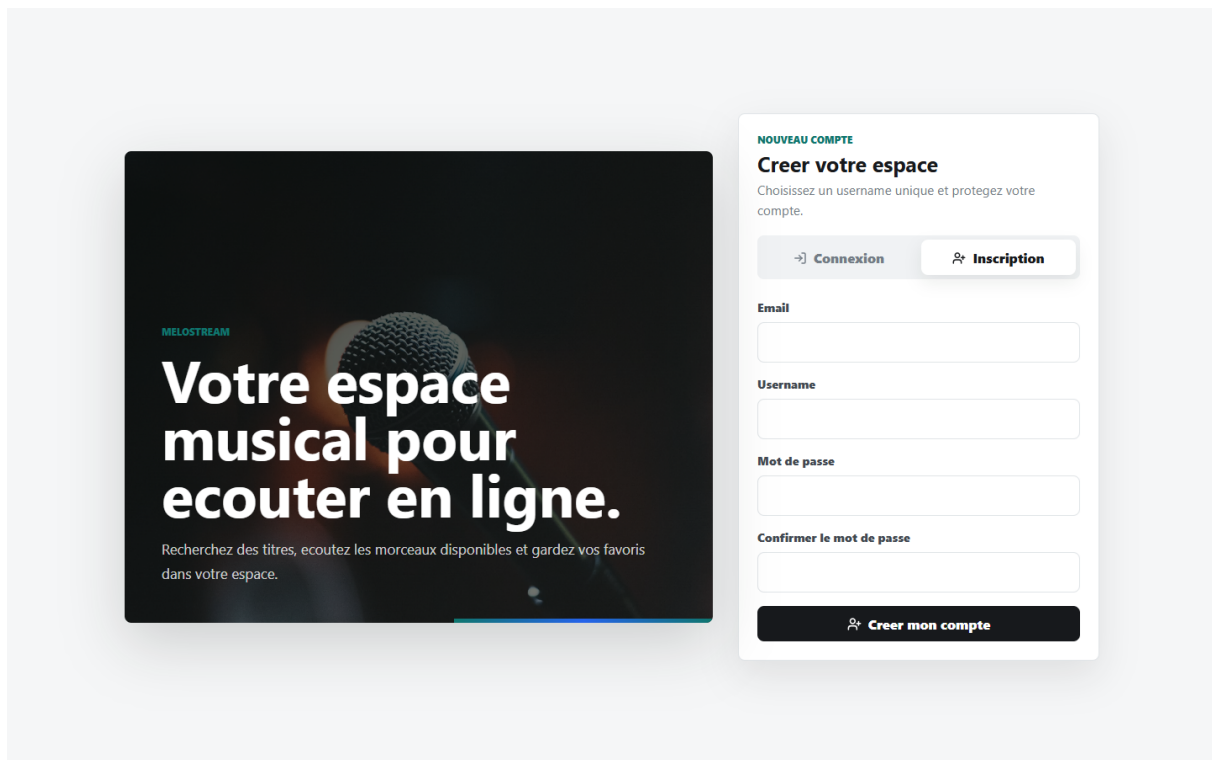


FIGURE 4.2 – Page d'inscription utilisateur

Explication de la figure : cette interface permet de créer un nouveau compte utilisateur. Les informations saisies sont envoyées au backend pour enregistrer le profil et générer une première session.

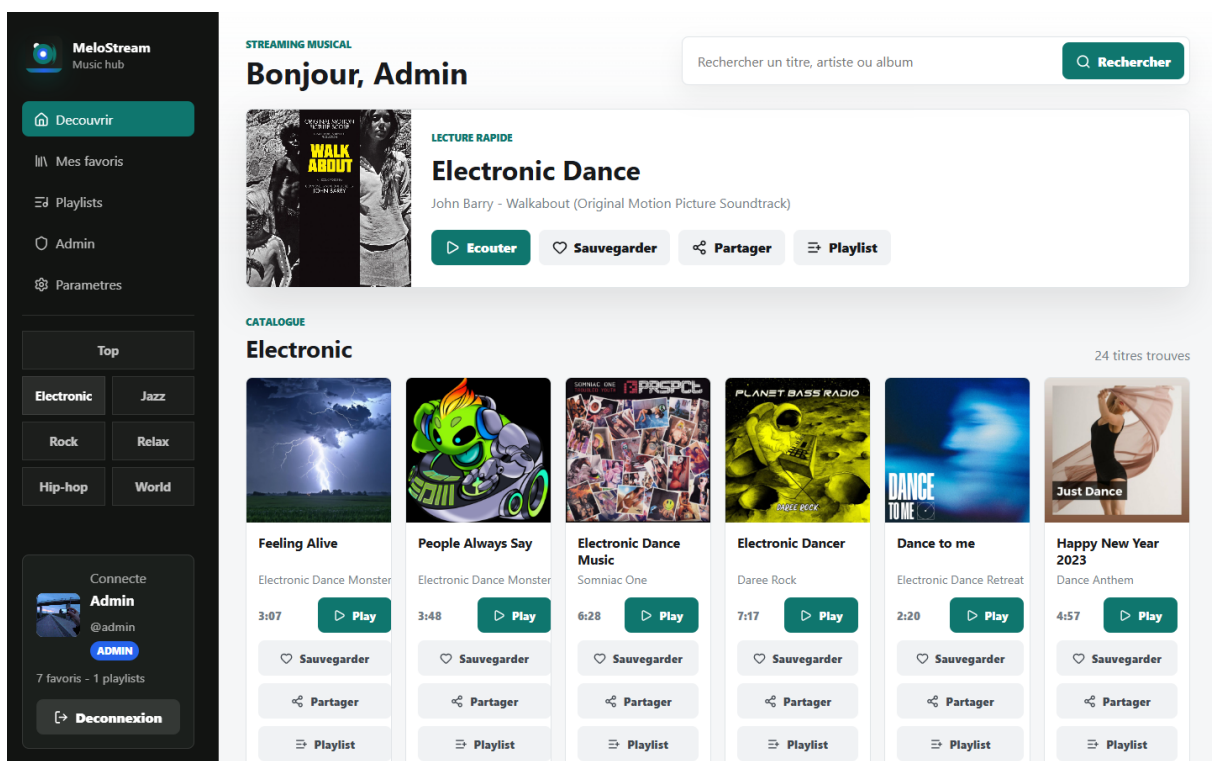


FIGURE 4.3 – Accueil utilisateur et catalogue musical

Explication de la figure : l'écran d'accueil présente le catalogue musical après connexion. L'utilisateur peut parcourir les titres proposés, lancer une écoute ou accéder aux actions rapides.

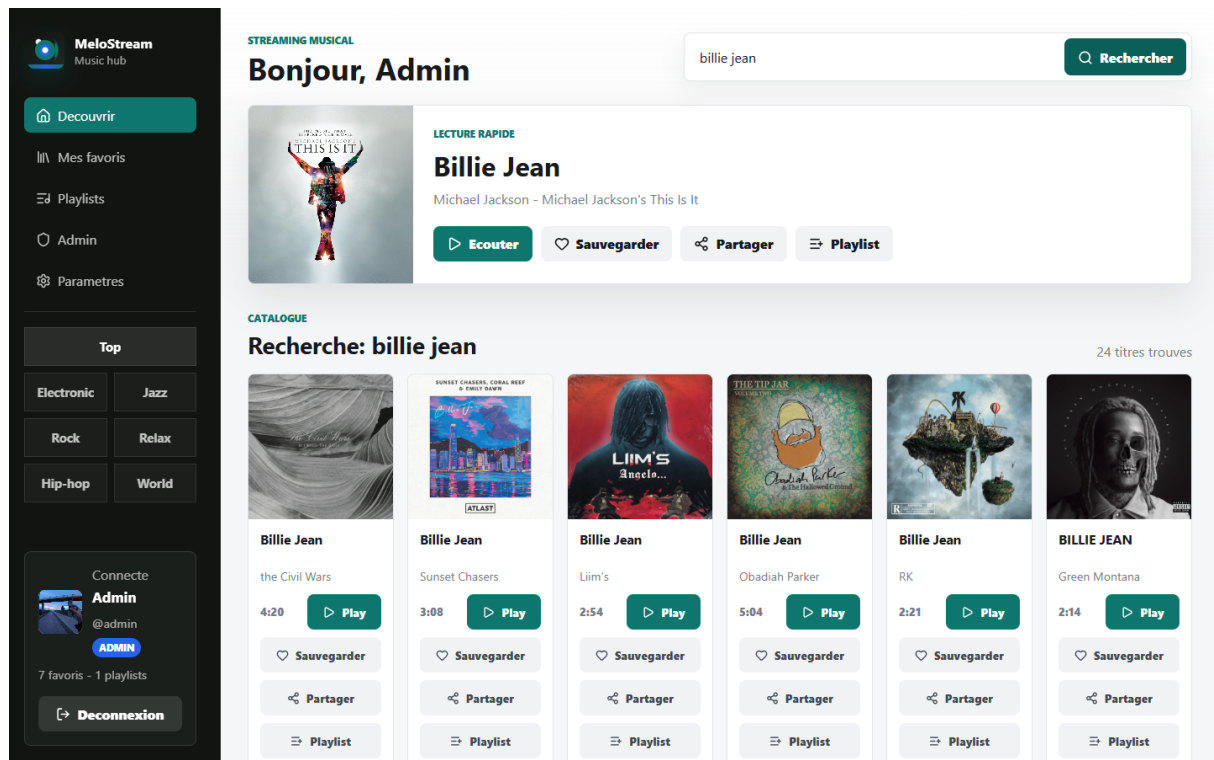


FIGURE 4.4 – Résultats d'une recherche de chanson

Explication de la figure : cette capture montre le résultat d'une recherche musicale. Les titres retournés par l'API sont affichés avec les informations nécessaires : artiste, album, image et actions disponibles.

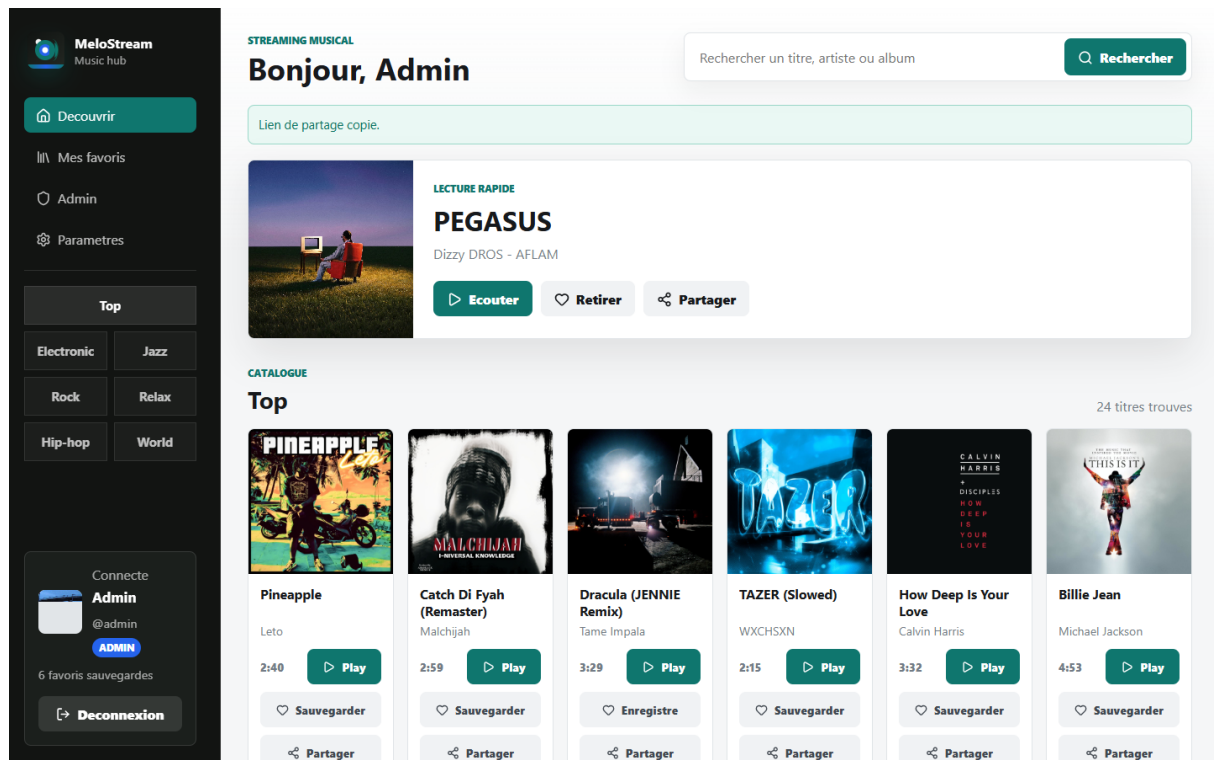


FIGURE 4.5 – Message affiché après le partage d'un titre

Explication de la figure : le message confirme que le lien de partage du titre a été généré. Ce lien peut être transmis à un autre utilisateur pour ouvrir directement le même morceau.

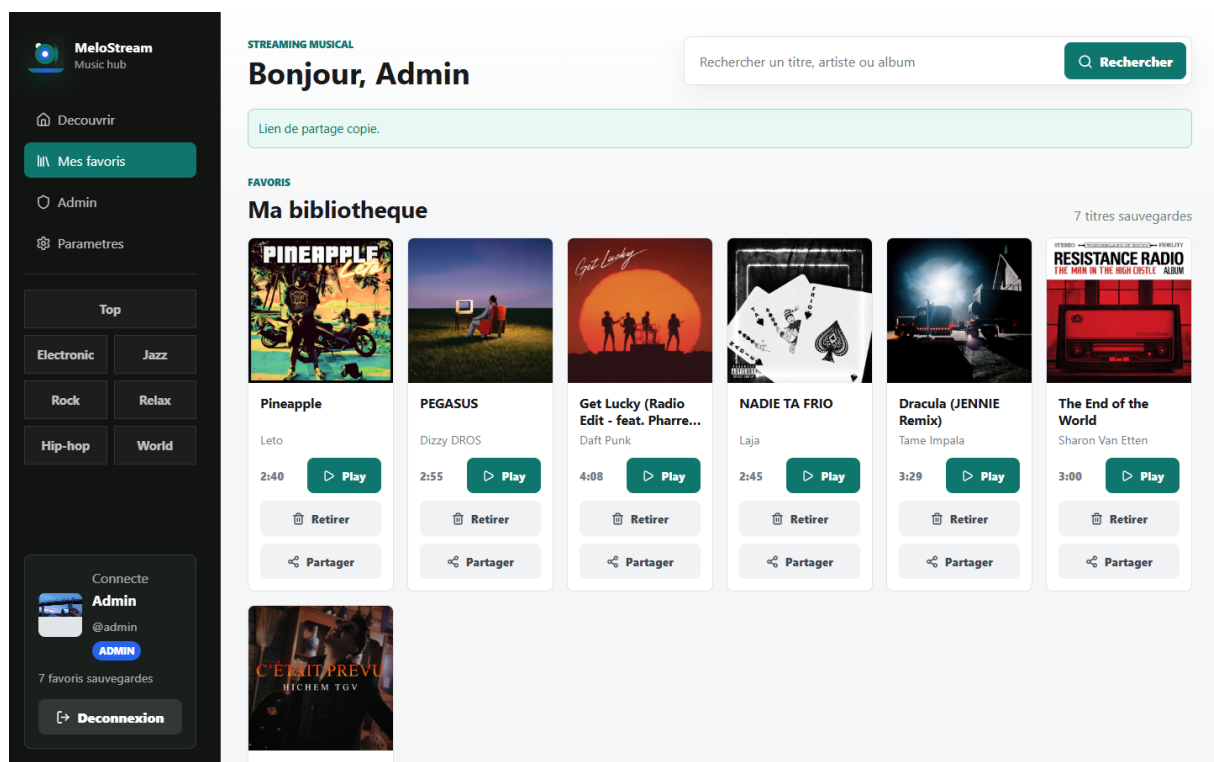


FIGURE 4.6 – Bibliothèque des titres favoris

Explication de la figure : cette page regroupe les titres sauvegardés par l'utilisateur. Elle permet de retrouver rapidement ses morceaux préférés et de les gérer depuis un espace personnel.

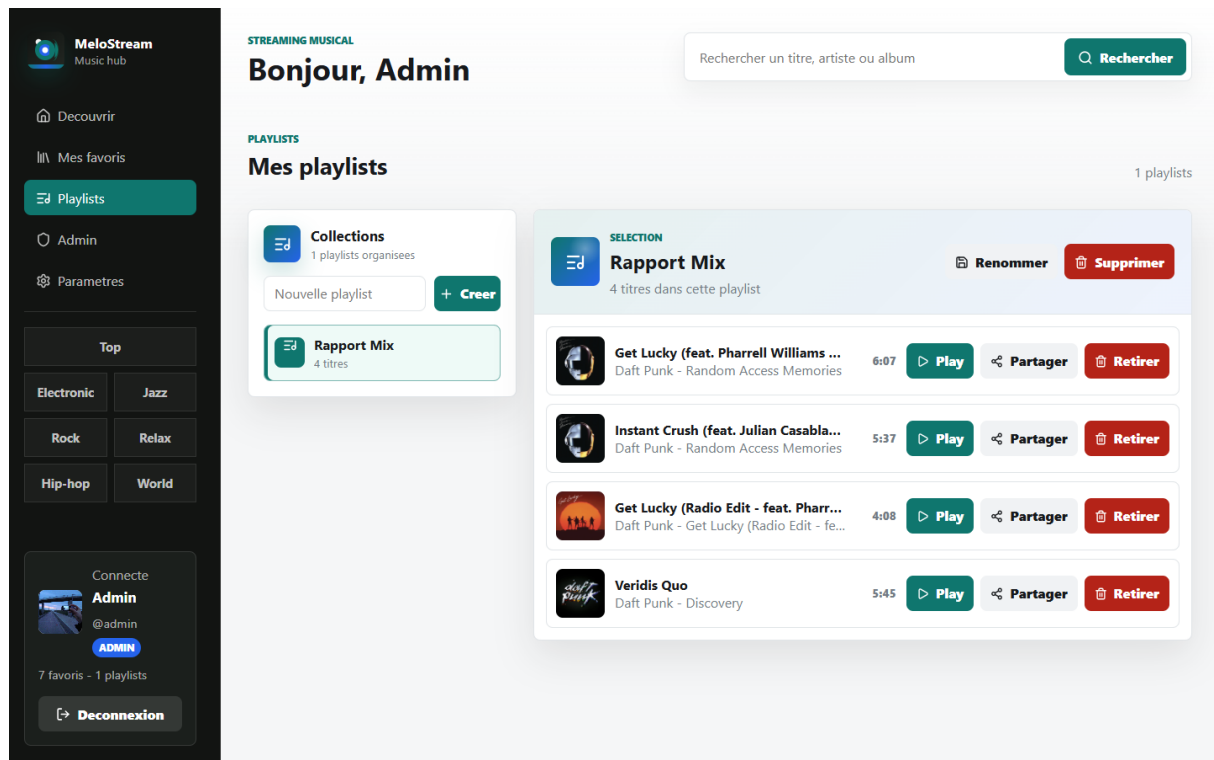


FIGURE 4.7 – Gestion des playlists utilisateur

Explication de la figure : l'écran des playlists permet d'organiser les chansons en listes personnalisées. L'utilisateur peut créer une playlist, y ajouter des titres ou retirer ceux qui ne sont plus souhaités.

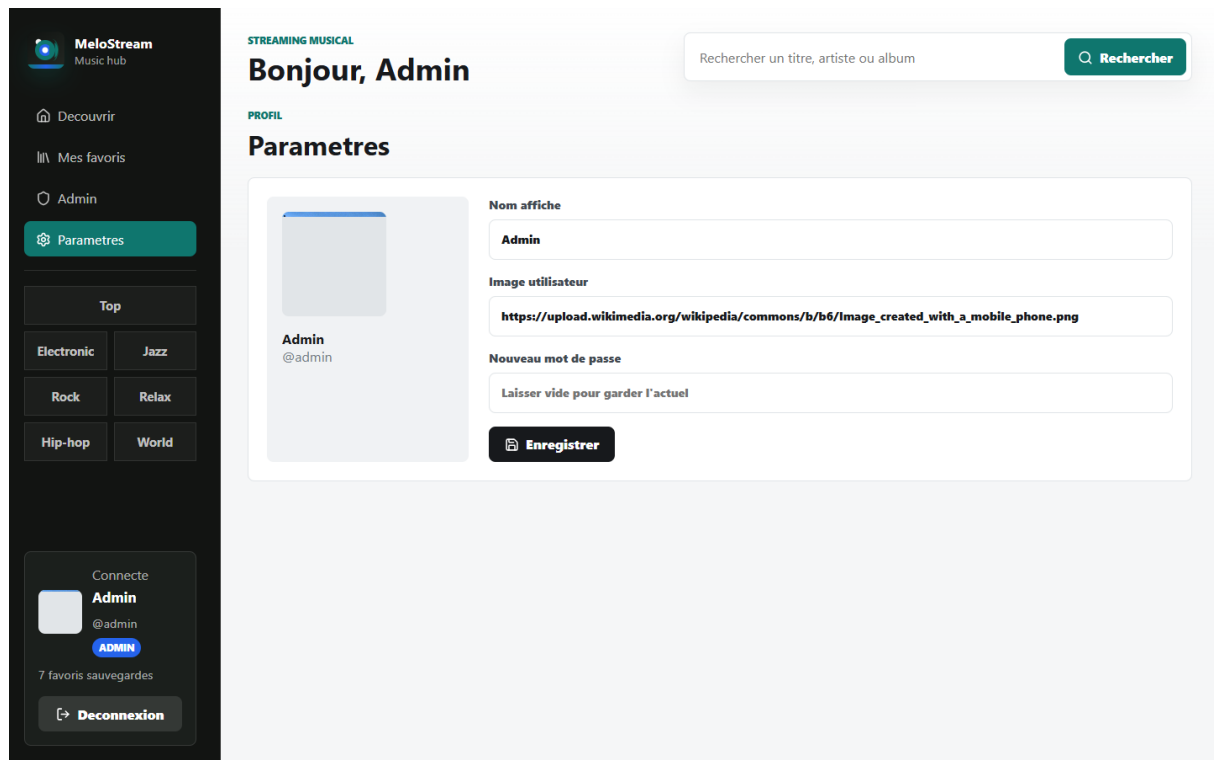


FIGURE 4.8 – Page des paramètres du profil

Explication de la figure : cette page permet de modifier les informations du profil, comme le nom d’affichage, l’avatar ou le mot de passe. Elle donne à l’utilisateur le contrôle sur ses données personnelles.

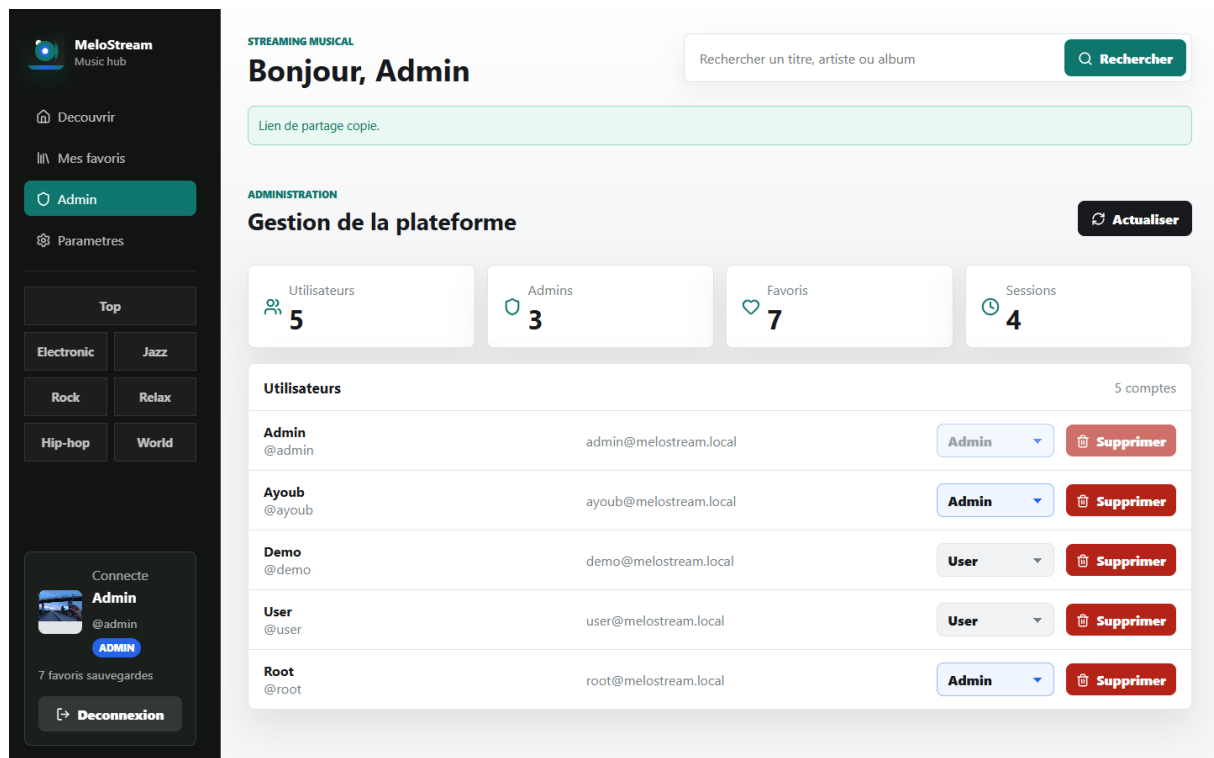


FIGURE 4.9 – Interface d’administration

Explication de la figure : cette interface est réservée aux comptes administrateurs. Elle permet de consulter les statistiques et de gérer les utilisateurs de l'application.

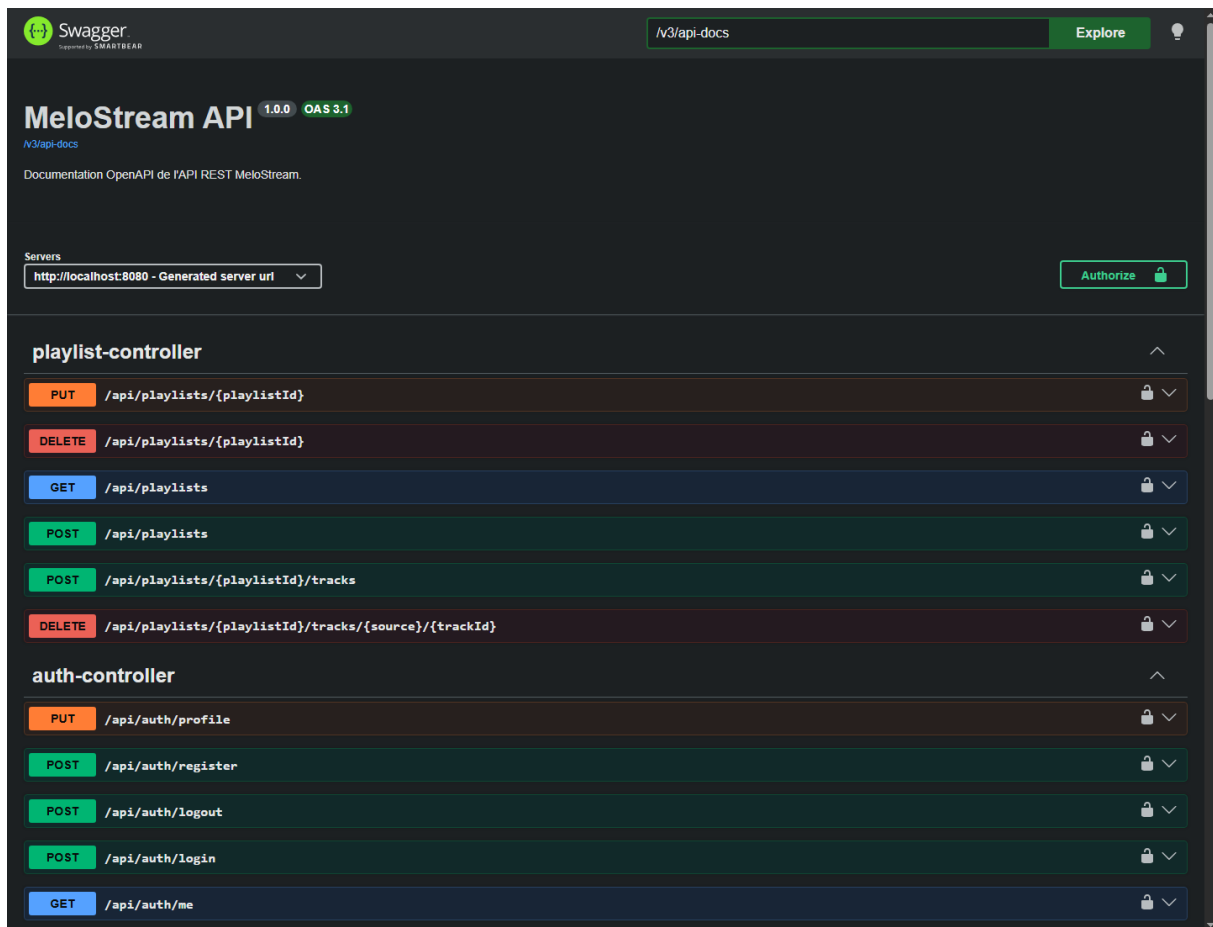


FIGURE 4.10 – Documentation Swagger OpenAPI de l'API REST

Explication de la figure : Swagger UI présente automatiquement la documentation des endpoints REST. Cette page facilite le test des routes et la compréhension des données échangées entre frontend et backend.

4.3 Comptes de démonstration

Email	Mot de passe	Rôle
admin@melostream.local	admin123	ADMIN
ayoub@melostream.local	ayoub123	USER
demo@melostream.local	demo123	USER
user@melostream.local	user123	USER
root@melostream.local	root123	ADMIN

Chapitre 5

Installation et lancement

5.1 Prérequis

- Java 21.
- Node.js et npm.
- MySQL local.
- Maven Wrapper inclus dans le backend.

5.2 Configuration de la base

La configuration par défaut utilise MySQL en local :

```
database: music_stream
username: root
password: vide
```

La base est créée automatiquement grâce à l'option `createDatabaseIfNotExist=true`.

Au premier démarrage, Hibernate crée ou met à jour les tables nécessaires selon les entités JPA du projet. Le composant de démarrage ajoute ensuite les comptes de démonstration si ceux-ci n'existent pas encore. Cette configuration simplifie les tests en local, car l'application peut être lancée sans script SQL manuel.

5.3 Lancement du backend

```
cd backend
.\mvnw.cmd spring-boot:run
```

Le backend démarre sur :

```
http://localhost:8080
```

La documentation Swagger est ensuite disponible sur :

```
http://localhost:8080/swagger-ui.html
```

5.4 Lancement du frontend

```
cd frontend  
npm install  
npm start
```

Le frontend démarre sur :

```
http://localhost:4200
```

5.5 Activation optionnelle de Jamendo

Pour utiliser Jamendo, il faut définir la variable d'environnement suivante avant de lancer le backend :

```
$env:JAMENDO_CLIENT_ID="ton_client_id_jamendo"
```

Chapitre 6

Vérification

6.1 Tests backend

```
cd backend
.\mvnw.cmd test
```

Les tests backend utilisent le profil de test et une base H2 en mémoire. Cela permet de vérifier le chargement du contexte Spring Boot et la configuration principale sans dépendre de la base MySQL locale.

6.2 Tests frontend

```
cd frontend
npm test -- --watch=false
```

6.3 Build frontend

```
cd frontend
npm run build
```

Le build frontend vérifie que l'application Angular se compile correctement, que les imports TypeScript sont valides et que les fichiers de production sont générés dans le dossier `dist`. Cette étape complète les tests backend et confirme que les deux parties du projet peuvent être lancées ensemble.

Conclusion

Le projet MeloStream répond aux objectifs fixés : il propose une plateforme musicale complète avec authentification, recherche, lecture, favoris, playlists, partage et administration. L'architecture sépare clairement le frontend Angular, le backend Spring Boot et la base MySQL, ce qui facilite la maintenance et les évolutions futures.

Les principaux défis rencontrés concernent l'intégration des API musicales externes, la gestion des liens de partage et la sécurisation des endpoints. Le projet dispose aussi d'une documentation Swagger/OpenAPI qui facilite la consultation et le test des routes REST. Il peut encore être amélioré avec une route Angular dédiée au partage, davantage de tests d'intégration et une configuration de déploiement adaptée à la production.